

of files or communication using odd ports.

Some statistical techniques are also available to find the expected number of hosts of a target network that would be explored by a user, or to compute the likelihood of a fumbling attack test that could either pass or fail.

Capturing TCP flags

In a UNIX environment, a *de facto* packet-capturing tool is *tcpdump*. It is powerful as well as flexible. As a UNIX tool, a powerful shell script is also applicable over the outputs of *tcpdump* and can produce a filtered report as desired. The underlying packet-capturing tool of *tcpdump* is *libcap* and it provides the source, destination, IP address, port and IP protocol over the target network interface for each network protocol. For example, to capture TCP SYN packets over the *eth0* interface, we can use the following command:

```
$ tcpdump -i eth0 "tcp[tcpflags] & (tcp-syn) != 0" -nn -v
```

Similarly, TCP ACK packets can be captured by issuing the command given below:

```
$tcpdump -I eth0 "tcp[tcpflags] & (tcp-ack) != 0" -nn -v
```

To have a combined capture report of SYN and ACK, both the flags can be combined as follows:

```
$tcpdump -I eth0 "tcp[tcpflags] & (tcp | tcp-ack) != 0" -nn -v
```

Getting network information

In this regard, *netstat* is a useful tool to get network connections, routing tables, interface statistics, masquerade connections, and multi-cast memberships. It provides a detailed view of the network to diagnose network problems. In our case, we can use this to identify ports that are listening. For example, to know about connections of HTTP and HTTPS traffic over TCP, we can use the following command expression with *-l* (to report socket), *-p* (to report relevant port) and *-t* (for only TCP) options.

```
$ netstat -tlp
```

Data analysis

Now, let's discuss a network data analysis example on *netstat* command outcomes. This will help you to understand the network traffic to carry out intrusion detection and prevention.

Let's say we have a csv file from the *netstat* command, as shown below:

```
> rfi <- read.csv("rficsv.csv",header=TRUE, sep=",")
```

...where the dimension, columns and object class are:

```
> dim(rfi)
```

```
[1] 302 11
> names(rfi)
[1] "ccadmin" "pts.0" "X" "ipaddress" "Mon" "Oct"
[7] "X30" "X17.25" "X.1" "still" "logged.in"
> class(rfi)
[1] "data.frame"
```

To make the relevant column heading meaningful, the first and fourth column headings are changed to:

```
> colnames(rfi)[1]='user'
> colnames(rfi)[4]='ipaddress'
```

If we consider a few selective columns of the data frame, as shown here:

```
> c = c(colnames(rfi)[1],colnames(rfi)[2],colnames(rfi)
[4],colnames(rfi)[5],colnames(rfi)[6],colnames(rfi)
[7],colnames(rfi)[8])
```

...then the first ten rows can be displayed to have a view of the table structure, as shown in Figure 1.

```
> x = rfi[, c(drop=F)]
> head(x,10)
  user    pts.0 ipaddress Mon Oct X30 X17.25
1  root    pts/1 172.16.7.226 Mon Oct 30 12:48
2 ccadmin pts/0 172.16.5.230 Mon Oct 30 12:30
3 ccadmin pts/0 172.16.5.230 Wed Oct 25 10:22
4  root    pts/1 172.16.7.226 Tue Oct 24 11:54
5  ccadmin pts/0 172.16.5.230 Tue Oct 24 11:53
6 (unknown :0      :0      Thu Oct 12 12:57
7  root    pts/0      :0      Thu Oct 12 12:57
```

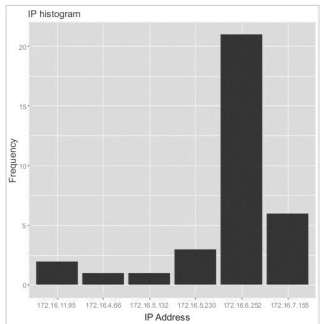


Figure 1: Histogram of IP addresses of *netstat*